File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

4. Parallelization Modes Explained (Simple English)

The Problem:

Matrix operations in quantum simulation involve:

- L matrix: size $2^n \times r$ (2^n rows, r columns)
- Gate application: update each row/column
- **Goal:** Make this faster using multiple CPU cores

Four Parallelization Strategies:

A. SEQUENTIAL (Baseline)

```
How it works:
- Process one gate at a time, one row at a time
- No parallelization at all
- Like one chef making burgers one at a time
```

Simple Explanation:

> "Do everything one step at a time. This is the baseline for comparison."

structured_csv.cpp +

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

Simple Explanation:

> "Do everything one step at a time. This is the baseline for comparison."

When it's fastest:

- Very small problems (n < 6)
- When OpenMP overhead > speedup benefit

B. BATCH Mode

```
How it works:
- Group gates into batches (e.g., 16 gates at once)
- Apply each batch to the state
- Uses the optimized simulator with truncation
```

Simple Explanation:

> "Instead of applying gates one-by-one, **collect multiple gates and apply them together** as a bigger operation. Like chopping all vegetables first, then cooking them all at once instead of chop-cook-chop-cook."

structured_csv.cpp +

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

B. BATCH Mode

```
How it works:
- Group gates into batches (e.g., 16 gates at once)
- Apply each batch to the state
- Uses the optimized simulator with truncation
```

Simple Explanation:

> "Instead of applying gates one-by-one, **collect multiple gates and apply them together** as a bigger operation. Like chopping all vegetables first, then cooking them all at once instead of chop-cook-chop-cook."

Analogy:

- Bad: Apply gate 1 → truncate → apply gate 2 → truncate (slow!)
- BATCH: Apply gates 1-16 → truncate once (faster!)

When it's best:

- Medium problems ($8 \leq n < 10$)
- When truncation overhead is high

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

Analogy:

- Bad: Apply gate 1 → truncate → apply gate 2 → truncate (slow!)
- BATCH: Apply gates 1-16 → truncate once (faster!)

When it's best:

- Medium problems ($8 \leq n < 10$)
- When truncation overhead is high
- When gates don't fuse well

C. ROW Mode

```
How it works:
1. Fuse consecutive gates on same qubit into one gate
   (e.g., H-X-H on q0 → combined matrix)
2. Apply gate to L matrix with row-level parallelism
   #pragma omp parallel for (over rows)
```

Simple Explanation:

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LREToS-docker-image 2 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

C. ROW Mode

```
How it works:
1. Fuse consecutive gates on same qubit into one gate
   (e.g., H-X-H on q0 → combined matrix)
2. Apply gate to L matrix with row-level parallelism
   #pragma omp parallel for (over rows)
```

Simple Explanation:

> "First, **combine multiple gates into one bigger gate** (fusion). Then apply that gate by splitting the work across CPU cores - **each core handles different rows** of the matrix. Like having 8 chefs each make burgers for different tables."

Example (Gate Fusion):

```
Before fusion: H → X → H on qubit 0
After fusion:  Single matrix M = H*X*H

Benefit: Apply M once instead of H, then X, then H (3x faster!)
```

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

Parallelism:

```
L matrix (2^10 = 1024 rows):
- Thread 0: processes rows 0-127
- Thread 1: processes rows 128-255
- Thread 2: processes rows 256-383
... (8 threads total)
```

When it's best:

- Large problems (n ≥ 10)
- Low-rank states (rank << 2^n) - typical for LRET!
- Sequential gates on same qubit (fuseable)

D. COLUMN Mode

```
How it works:
- Apply gates with column-level parallelism
  #pragma omp parallel for (over columns)
```

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image 2 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

D. COLUMN Mode

```
How it works:
- Apply gates with column-level parallelism
  #pragma omp parallel for (over columns)
- Each thread handles different columns
```

Simple Explanation:

> "Like ROW mode, but **each CPU core handles different columns** instead of rows. This works better when the matrix has many columns (high-rank states)."

Visual:

```
L matrix (rows=1024, columns=64):

ROW mode splits by rows:
[Row 0-127   ]  ← Thread 0
[Row 128-255]  ← Thread 1
```

structured_csv.cpp +

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LREToS-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

```
L matrix (rows=1024, columns=64):

ROW mode splits by rows:
[Row 0-127  ]  ← Thread 0
[Row 128-255]  ← Thread 1
...

COLUMN mode splits by columns:
[Col 0-7 | Col 8-15 | Col 16-23 | ...]
   ↑         ↑           ↑
 Thread0   Thread1    Thread2
```

When it's best:

- High-rank states (rank >> 1)
- When `--initial-rank` is large
- When memory access patterns favor column-wise operations

F. HYBRID Mode

structured_csv.cpp +

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LREToS-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

E. HYBRID Mode

```
How it works:
1. Build "layers" of non-overlapping gates
   Layer 1: [Gate on q0, Gate on q2, Gate on q4] (can run in parallel!)
   Layer 2: [Gate on q1, Gate on q3]
2. Apply all gates in a layer simultaneously (layer parallelism)
3. Within each gate, use row/column parallelism
```

Simple Explanation:

> "Find gates that operate on **different qubits** and apply them **at the same time** (they don't interfere). Like having multiple chefs cook different dishes simultaneously - one makes burgers (q0), another makes pizza (q2), another makes salad (q4) - all at once!"

Example:

```
Circuit:
H(q0) → X(q1) → H(q0) → CNOT(q0,q1)

Layers:
```

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

Example:

```
Circuit:
H(q0) → X(q1) → H(q0) → CNOT(q0,q1)

Layers:
Layer 1: [H(q0), X(q1)]  ← Applied in parallel (different qubits!)
Layer 2: [H(q0)]         ← Must wait for Layer 1
Layer 3: [CNOT(q0,q1)]   ← Must wait for Layer 2
```

When it's best:

- Deep circuits with many qubits
- Sparse gate placement (gates on different qubits)
- When you want to maximize CPU utilization

Performance Comparison Table:

Mode	Best For	Why	Typical Speedup

structured_csv.cpp

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++

File Edit Selection View Go ··· intern

TERMINAL CHAT

EDITING README TO INCLUDE DOCKER IMAGE INSTRUCTIONS

When it's best:

- Deep circuits with many qubits
- Sparse gate placement (gates on different qubits)
- When you want to maximize CPU utilization

Performance Comparison Table:

Mode	Best For	Why	Typical Speedup
SEQUENTIAL	n < 6	OpenMP overhead too high	1.0x (baseline)
BATCH	8 ≤ n < 10	Reduces truncation overhead	1.2-1.5x
ROW	n ≥ 10, low rank	Row parallelism + fusion	2-4x
COLUMN	High rank states	Column parallelism	1.5-2.5x
HYBRID	Deep, sparse circuits	Layer + row/column parallelism	2-3x

Real-World Analogy Summary:

Mode	Restaurant Analogy
SEQUENTIAL	One chef makes one burger at a time

structured_csv.cpp +

Describe what to build next

Agent Claude Sonnet 4.5

Branch-after-LRETo5-docker-image ⊗ 2 ⚠ 0 Ln 1, Col 1 Spaces: 4 C++